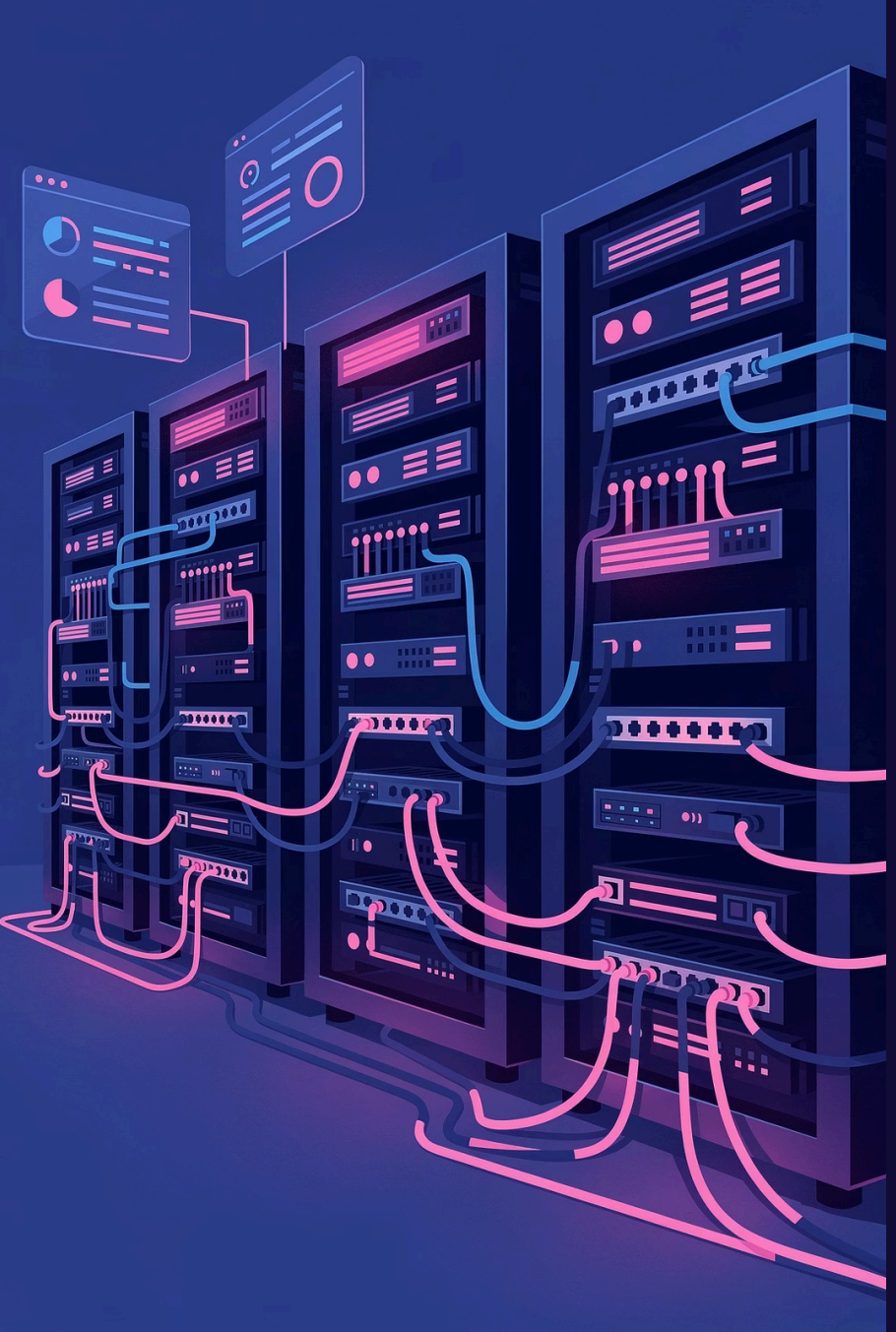


Session 4: Extending Our AI Backend Service

Applied Generative AI Engineering · Shrey, AI Solution Engineer

MODULE 2 · PYTHON API MIDDLEWARE LAYER

Course Progress

Session 4 marks the **completion of Module 2** -the Python API Middleware Layer. You've built the foundation; today you make it production-ready.

01

Module 1 – AI Foundations

LLMs, prompts, API calls ✔️

02

Module 2 – Python API Middleware

Current — completing today

03

Module 3 – RAG Systems

Retrieval-augmented generation (next)

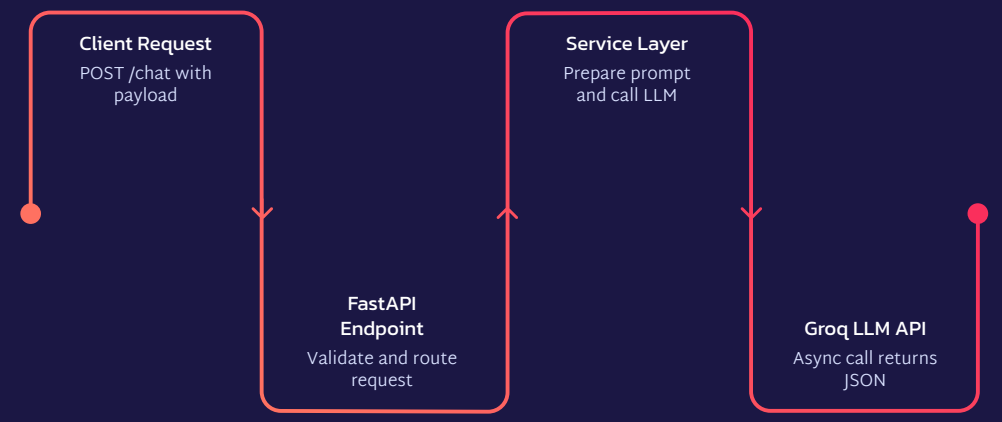
04

Modules 4–6

RAG Optimisation · Fine-Tuning · Agentic Systems

Recap: What You Built in Session 3

Architecture Built



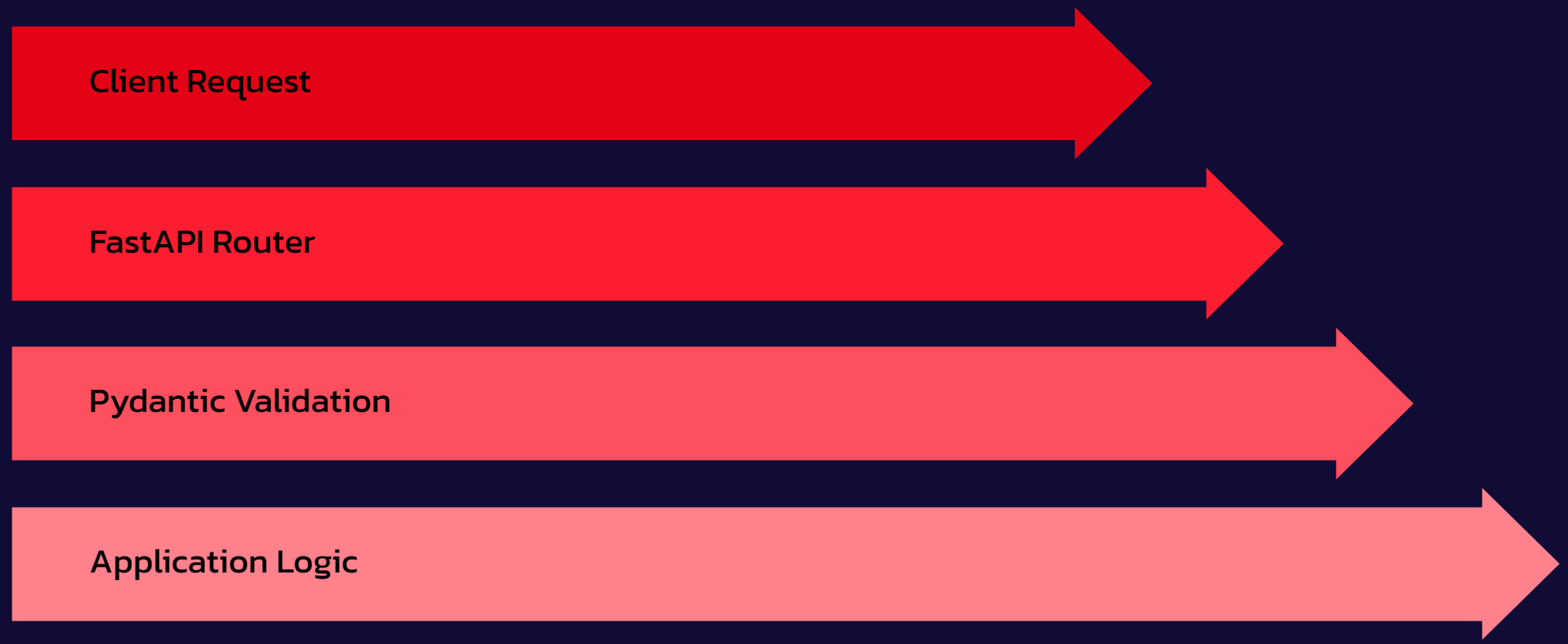
Key Concepts Covered

- Exposing AI logic via `POST /chat` endpoint
- FastAPI request lifecycle and ASGI runtime
- Async LLM calls with `httpx` / Groq SDK
- Pydantic request validation and JSON responses
- Serving with Uvicorn; testing via Swagger UI
- Modular project structure: routers / services / models

Today we **extend** this architecture - not rebuild it. Everything you wrote is the starting point.

FastAPI Request Lifecycle

Understanding the full request pipeline is essential before we extend it. Every layer has a distinct responsibility - and today we're adding production concerns at each one.



Each stage is a potential failure point. Session 4 introduces **error handling, logging, and provider abstraction** directly into this pipeline.

Current Codebase Architecture

Project Structure

```
app/
 ├── main.py
 ├── routers/
 │    └── chat.py
 ├── services/
 │    └── llm.py
 └── models/
      └── request.py
```

Layer Responsibilities

main.py

App entry point, middleware registration, router mounting

routers/chat.py

Route definitions, request intake, response formatting

services/llm.py

Business logic - constructs and dispatches LLM calls

models/request.py

Pydantic schemas - enforces input/output contracts

The Endpoint You Implemented

POST /chat

```
@router.post("/chat")
async def chat(req:
ChatRequest):
    response = await
llm_service.call(
        prompt=req.prompt
    )
    return {"response": response}
```

What It Does Well

- **Async LLM call** — non-blocking, ASGI-compatible
- **Pydantic validation** — type-safe request handling
- **Clean JSON response** — consistent API contract
- **Separation of concerns** — router delegates to service

This is a solid prototype. But it has critical gaps before it can serve real production traffic.

Is This Production-Ready?

Before extending the system, we need to stress-test its assumptions. Ask yourself:

Provider Failure

What happens when the **Groq API returns a 429 or 503**? Does your backend crash, hang, or recover gracefully?

Provider Switching

A client requests **OpenAI GPT-4** instead of Groq. How much code do you have to change right now?

Multi-Model Support

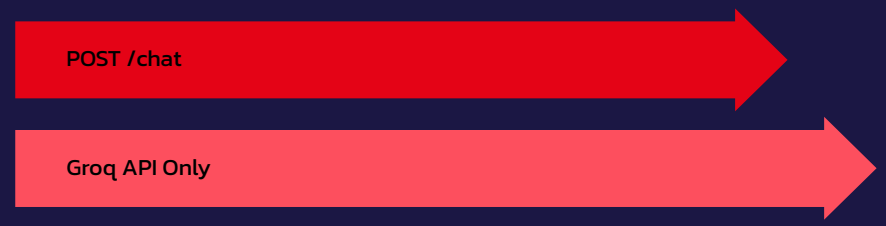
Your system needs to support **three different models simultaneously**. How does the current architecture scale?

Credential Exposure

Is your **API key committed to source control**? Is it injectable per environment without code changes?

The Hardcoded Provider Problem

Current Coupling



The service layer **directly imports and calls** the Groq SDK — the provider is baked into the business logic.

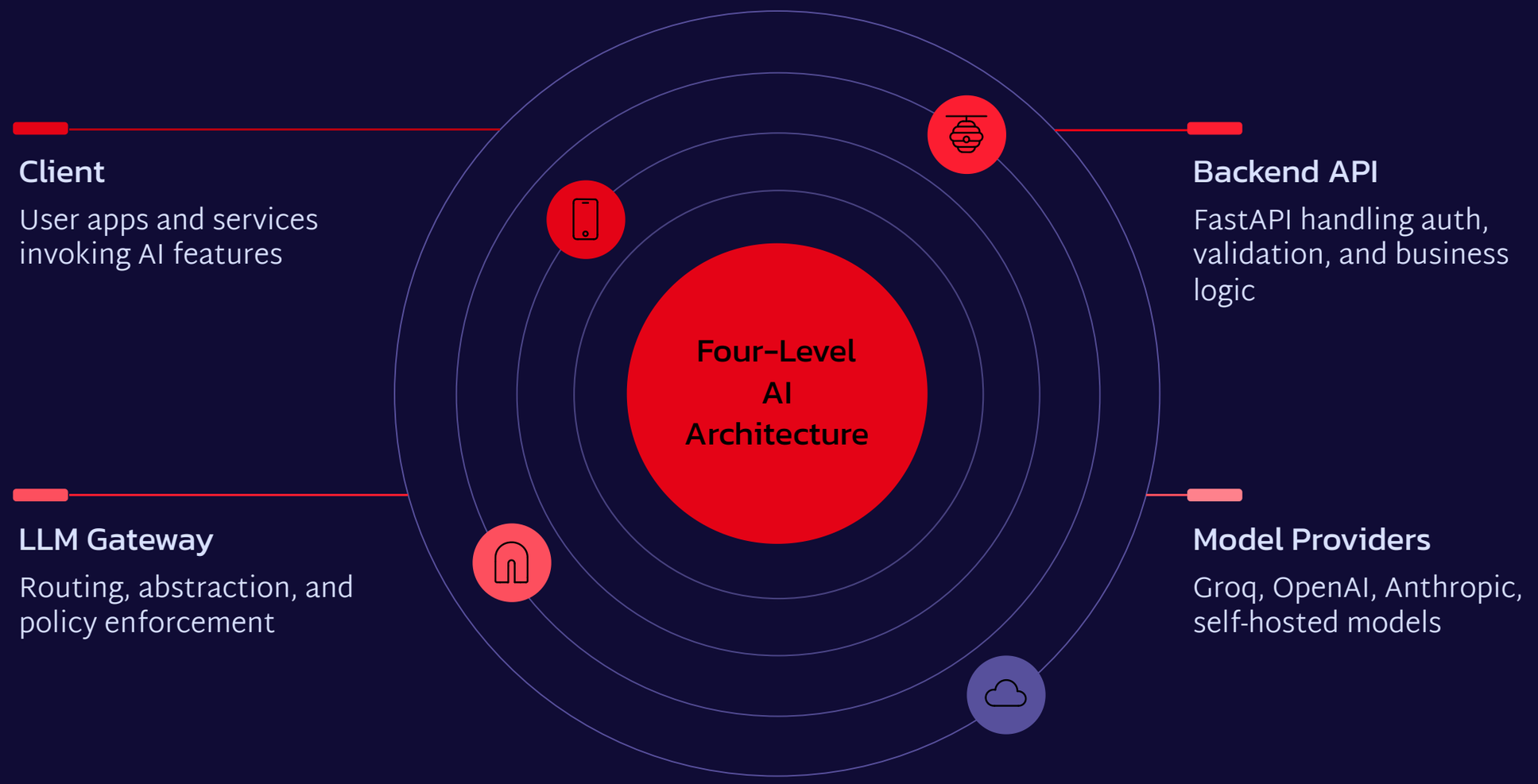
Why Vendor Lock-In Is Dangerous

- **A single provider outage** takes your entire service offline
- **Switching providers** requires rewriting service logic and retesting everything
- **Adding a second model** forces duplicated code or messy conditionals
- **Cost optimisation** (routing cheap requests to faster providers) is impossible
- **Compliance requirements** may mandate specific providers per data type

Tightly coupled provider code is a single point of failure, not a feature.

Production AI Backend Architecture

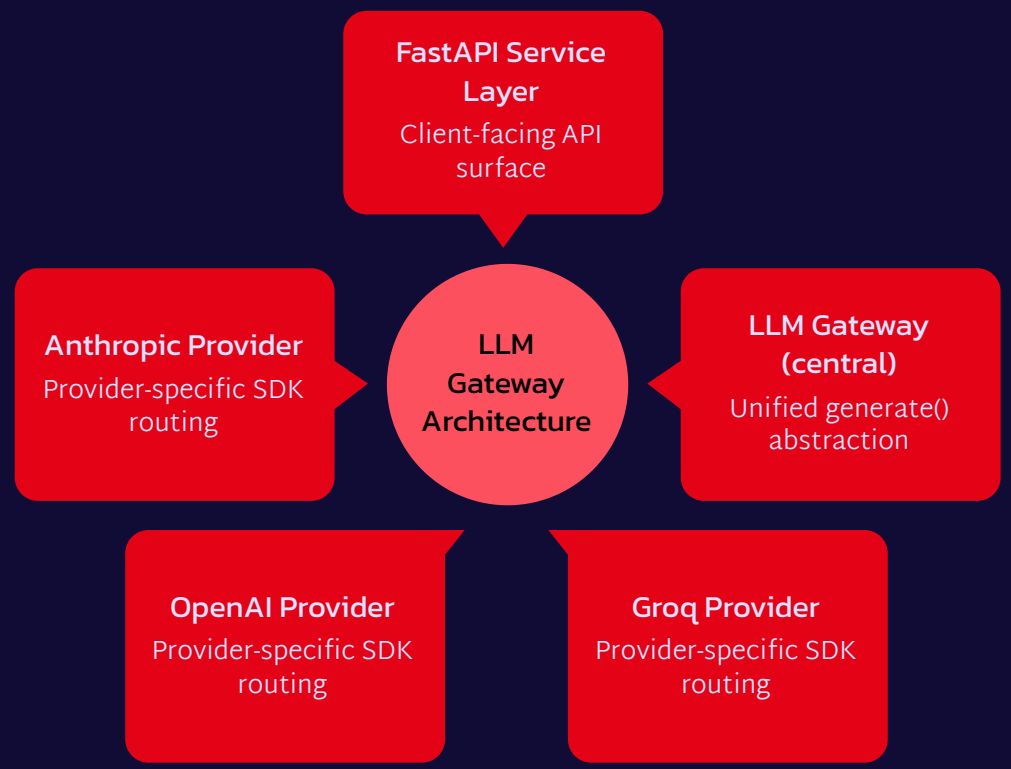
Real-world AI platforms introduce a **gateway layer** between the application and the model providers — decoupling routing logic from business logic entirely.



The gateway becomes the **single point of control** for all LLM interactions: provider selection, retry logic, key management, and observability all live here - invisible to the rest of the application.

Introducing the LLM Gateway

Gateway Architecture



What the Gateway Abstracts

Unified Interface

Callers invoke a single `generate()` method regardless of provider

Provider Routing

Gateway reads the `provider` field and dispatches to the correct SDK

Credential Injection

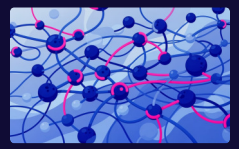
Each provider's API key is resolved from environment variables at runtime

Error Isolation

Provider failures are caught and normalised before reaching the router

Why Multiple LLM Providers Exist

No single provider is optimal for every workload. Production systems select the right model for each task based on latency, cost, context window, and compliance needs.

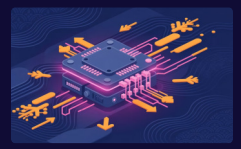


OpenAI

Use case: Strong general reasoning, function calling, structured output generation

Anthropic Claude

Use case: Long-context document analysis, nuanced instruction following, safety-sensitive applications



Groq

Use case: Ultra-low-latency inference via custom LPU hardware - ideal for real-time and streaming use cases

Self-Hosted Models

Use case: Air-gapped or privacy-sensitive workloads where data must never leave your infrastructure

Gateway Design Concept

The Core Abstraction

```
# The only interface callers need
async def generate(
    provider: str,
    prompt: str,
    model: str | None = None,
    **kwargs
) -> str:
    ...

# The rest of the app never
# knows which SDK runs below
```

The gateway is the **only place** in the codebase that imports provider-specific SDKs.

Design Principles Applied

Dependency Inversion

High-level modules (routers, services) depend on the abstraction — not on Groq or OpenAI directly

Open/Closed Principle

Adding a new provider means adding a new handler function — existing code is not modified

Single Responsibility

Provider dispatch, credential resolution, and error normalisation all belong exclusively to the gateway

Gateway Class Design

The gateway is implemented as a Python class with a dispatcher pattern. Each provider maps to a dedicated async handler method.

```
class LLMGateway:
    def __init__(self):
        self.providers = {
            "groq": self._call_groq,
            "openai": self._call_openai,
            "anthropic": self._call_anthropic,
        }

    async def generate(
        self, provider: str, prompt: str
    ) -> str:
        handler = self.providers.get(provider)
        if not handler:
            raise ValueError(
                f"Unknown provider: {provider}"
            )
        return await handler(prompt)

    async def _call_groq(self, prompt: str) -> str:
        # Groq SDK call here
        ...

    async def _call_openai(self, prompt: str) -> str:
        # OpenAI SDK call here
        ...
```

Key Design Decisions

- **Dictionary dispatch** avoids long `if/elif` chains and is trivially extensible
- **Unknown provider** raises a typed `ValueError` — caught upstream by error handling middleware
- **Each handler is async** — the gateway is fully non-blocking and ASGI-safe
- **Single instantiation** — the gateway is created once and injected via FastAPI's dependency system

The updated project structure will add `services/gateway.py` to house this class.

Updating the API Endpoint

New Request Schema

```
# models/request.py
class ChatRequest(BaseModel):
    provider: str = "groq"
    model: str | None = None
    prompt: str
    temperature: float = 0.7
```

Updated Endpoint

```
# routers/chat.py
@router.post("/chat")
async def chat(
    req: ChatRequest,
    gateway: LLMGateway = Depends(get_gateway)
):
    response = await gateway.generate(
        provider=req.provider,
        prompt=req.prompt
    )
    return {"response": response}
```

What Changed — and Why

provider field added

Clients now declare which provider to use. Defaults to `"groq"` for backward compatibility.

model field added

Optional override — useful when a provider offers multiple model versions (e.g. `gpt-4o` vs `gpt-4o-mini`).

Dependency injection

Gateway is injected via `Depends()` — clean, testable, and mockable in unit tests.

Environment Variable Configuration

.env File (never commit this)

```
# .env
OPENAI_API_KEY=sk-...
GROQ_API_KEY=gsk_...
ANTHROPIC_API_KEY=sk-ant-...
DEFAULT_PROVIDER=groq
LOG_LEVEL=INFO
```

Loading with python-dotenv

```
from dotenv import load_dotenv
import os

load_dotenv()

OPENAI_KEY = os.getenv("OPENAI_API_KEY")
GROQ_KEY   = os.getenv("GROQ_API_KEY")
```

Secrets Management Principles

Never hardcode API keys

Keys in source code will eventually be leaked via git history, logs, or error messages

.env goes in .gitignore

Use `.env.example` with placeholder values as documentation for new developers

Use secret managers in production

AWS Secrets Manager, GCP Secret Manager, or HashiCorp Vault replace `.env` files at scale

Error Handling for LLM Calls

LLM APIs fail in ways that are different from standard HTTP services. Your backend must handle these failures **without crashing or exposing internals** to the client.

Common Failure Scenarios

Timeouts

LLM generation can take 10–30 seconds. Always set explicit `timeout` values on HTTP clients.

Rate Limits (429)

Providers throttle requests per minute. Implement exponential backoff with jitter.

Provider Outages (503)

Return a clear `503 Service Unavailable` upstream rather than an unhandled exception.

Error Handling Pattern

```
async def _call_groq(self, prompt: str) -> str:
 try:
 response = await client.chat(prompt)
 return response.text
 except RateLimitError:
 raise HTTPException(
 status_code=429,
 detail="Provider rate limit hit"
 )
 except TimeoutError:
 raise HTTPException(
 status_code=504,
 detail="LLM provider timed out"
 )
 except Exception as e:
 logger.error(f"LLM error: {e}")
 raise HTTPException(status_code=502)
```

Logging for Backend Observability

Structured Logging Setup

```
import logging

logging.basicConfig(
    level=os.getenv("LOG_LEVEL", "INFO"),
    format="%(asctime)s %(levelname)s "
           "%(name)s: %(message)s"
)
logger = logging.getLogger(__name__)

# In the gateway:
logger.info(
    f"LLM request | provider={provider} "
    f"prompt_len={len(prompt)}"
)
logger.info(
    f"LLM response | latency={elapsed:.2f}s"
)
logger.error(
    f"LLM failure | provider={provider} "
    f"error={str(e)}"
)
```

Three Reasons to Log Every LLM Call

Request Tracing

Correlate a slow response or error to a specific provider, model, and prompt — essential for debugging in production

Debugging

Structured logs let you filter by `provider`, `status`, or `latency` without trawling raw output

Production Monitoring

Logs feed dashboards and alerting systems — you can track error rates and latency trends over time

Module 2 Complete — What You've Built

FastAPI Architecture

Modular, layered backend with clean separation between routing, service logic, and data models

Provider Abstraction

LLM Gateway decouples the application from any specific SDK — new providers are added with zero changes to existing code

Credential Management

API keys loaded from environment variables — ready for deployment to staging and production environments

Error Handling

Graceful handling of timeouts, rate limits, and provider failures with correct HTTP status codes

Logging

Structured request/response logging enabling tracing, debugging, and production monitoring

The Limitation We Haven't Solved Yet

The backend you've built is a powerful, flexible LLM gateway — but it only knows what the model was **trained on**. It cannot reason over information that lives outside the model.

Private Documents

Internal PDFs, reports, policies, and procedures that were never part of any model's training data

Company Knowledge Bases

Confluence wikis, Notion databases, internal handbooks — live institutional knowledge the model has never seen

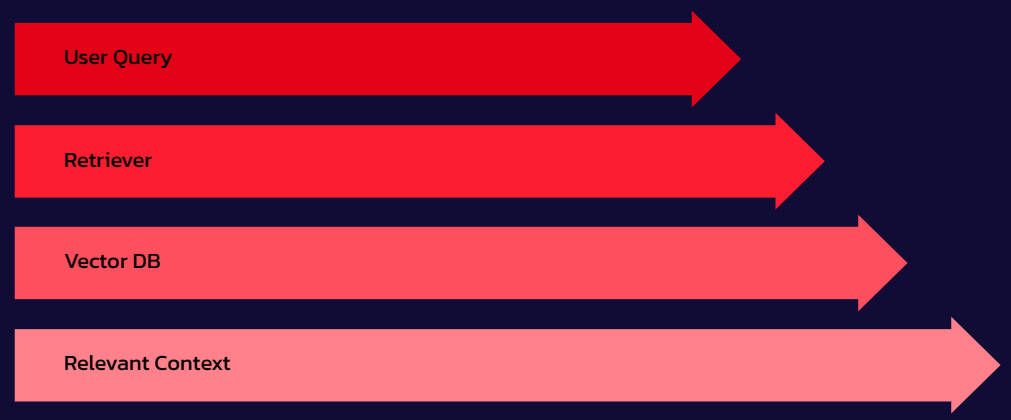
Live External Data

Real-time information, recent events, or any content published after the model's training cut-off date

The gap between "what the model knows" and "what your organisation knows" is the problem Module 3 solves.

Module 3 Preview: Retrieval-Augmented Generation

RAG Architecture



What's Coming Next

Vector Embeddings

Convert documents into dense numerical representations that capture semantic meaning

Vector Databases

Store and query embeddings at scale using tools like `Chroma`, `Pinecone`, or `pgvector`

Context Injection

Retrieved document chunks are injected into the prompt — grounding the LLM in your private data

Knowledge-Aware Backends

Your FastAPI gateway becomes the backbone of a **question-answering system over private documents**